Final Examination Report

Retail Return Risk Prediction Using Structured Data and NLP

Student Name:	Robert Tumushiime
Access Number:	B31332
Programme:	MSc Data Science
Course and Year:	Year 2, Semester 1
Course Unit:	Data Mining, Modelling and Analytics
Submission Date:	19 March 2026

Submitted Requirements Included:

Notebook analysis, cleaned dataset, saved model artefact, predictions, NLP outputs, deployment scripts, and this final report with application link and user guidelines.

1 Executive Summary

This project addresses a practical retail analytics problem: predicting whether a transaction line is likely to be returned or cancelled before dispatch. The solution combines structured transaction attributes (for example quantity, unit price, temporal fields, and country) with text mining over product descriptions. A complete pipeline was built covering data acquisition, cleaning, feature engineering, model comparison, threshold optimization, NLP analysis, and deployment design.

The deployed Streamlit Cloud application uses in-app model inference as its backend, so no external API service is required for prediction. The selected production model is a HistGradientBoosting classifier saved in a joblib artefact together with an optimized decision threshold and feature metadata. Because the return class is highly imbalanced, model quality was evaluated with ROC-AUC and PR-AUC, with operational emphasis on return-class recall and F1 at the optimized threshold.

The published deployment is therefore a single-tier architecture: users interact with the Streamlit interface, and the same app process performs preprocessing and model inference.

2 Problem Definition and Business Objective

Online retail operations frequently lose margin through returns, cancellations, reverse logistics, and handling costs. The analytical objective is to estimate return risk at transaction level using historical sales records and product-description text.

Target variable:

- ReturnFlag = 1 if invoice number indicates cancellation (prefix C), else 0.

Business objective:

- Predict risk before shipment.
- Support actions such as manual review, product-page refinement, and customer communication.
- Reduce cost of missed returns while preserving normal sales throughput.

3 Dataset and Preparation

The analysis used the UCI Online Retail dataset, containing over half a million transaction lines from an e-commerce business. The data contains both structured and unstructured signals:

- Structured: Quantity, UnitPrice, InvoiceDate, CustomerID, Country, StockCode.

- Unstructured: Description text.

Cleaning and preparation followed an exam-ready reproducible process:

1. Remove duplicates.
2. Drop rows with missing critical fields.
3. Standardize text fields and normalize descriptions.
4. Remove invalid transaction values (UnitPrice ≤ 0, Quantity $= 0$).
5. Treat outliers through percentile clipping.
6. Derive engineered features and save cleaned output as `Tumushiime.csv`.

Figure 1 summarizes distributional characteristics and class imbalance in the cleaned dataset.

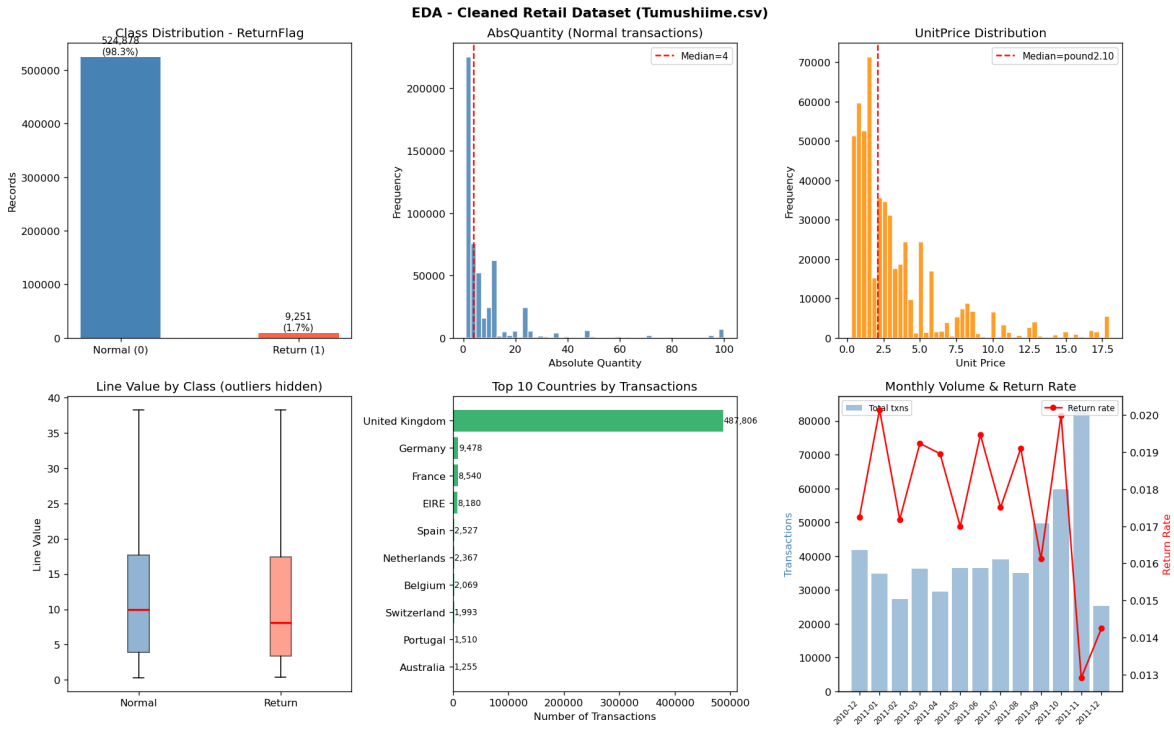

Figure 1: EDA summary for cleaned retail dataset.

4 Feature Engineering and Modelling

4.1 Engineered Features

The final model uses a combined feature space:

- Numeric and categorical engineered features (9 fields): UnitPrice, AbsQuantity, LineValueAbs, InvoiceMonth, InvoiceWeekday, InvoiceHour, CustomerIDMissing, DescriptionLength, CountryEncoded.

- Text representation: TF-IDF with 200 features (unigrams and bigrams).

Random Forest importance analysis was used to validate useful predictors and explain feature contributions.

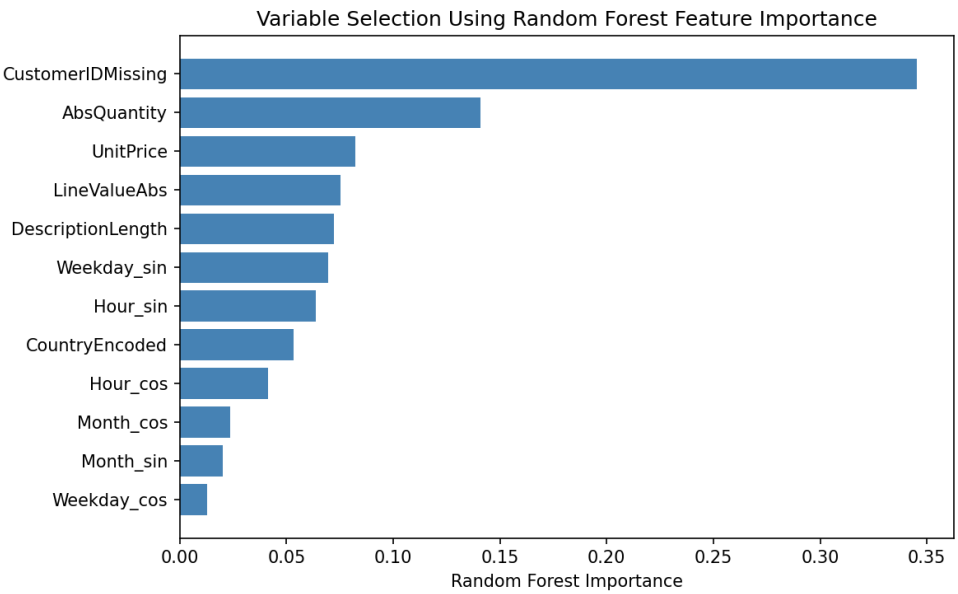

Figure 2: Numeric feature importance for variable selection and interpretation.

4.2 Model Comparison and Selection

Three classifiers were trained and compared on a held-out test set:

- Logistic Regression (baseline)
- Random Forest
- HistGradientBoosting (selected best model)

Since returns are rare, the primary evaluation metrics were ROC-AUC and PR-AUC. A probability threshold was tuned to improve return-class F1 for operational deployment.

Table 1: Model comparison summary (actual notebook outputs).

Model	ROC-AUC	PR-AUC	Return Recall	Return F1
Logistic Regression	0.7445	0.0521	0.7135	0.0632
Random Forest	0.8371	0.1471	0.6822	0.1170
HistGradientBoosting	0.8594	0.2159	0.7459	0.1146

The selected production model is HistGradientBoosting because it achieved the best ROC-AUC and PR-AUC. The saved deployment threshold is 0.82499 from the model artefact.

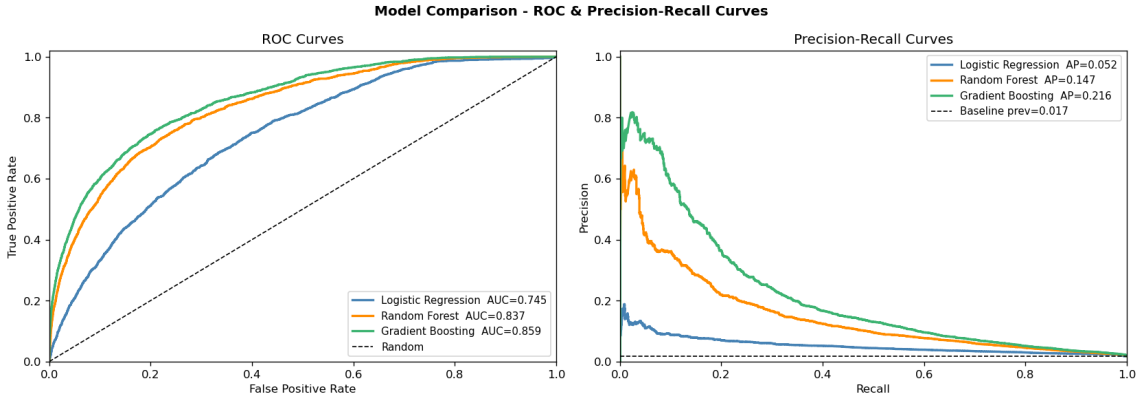

Figure 3: ROC and Precision-Recall curves for model comparison.

Final model artefact saved: `Tumushiime best model.joblib`.

5 NLP Analysis and Insights

To evaluate independent text signal, a text-only experiment was performed using product description text. The workflow used term frequency analysis, TF-IDF features, and Logistic Regression.

Key findings:

- Product text provides meaningful predictive signal for return behavior.
- Administrative terms (for example manual, postage, adjustment-like language) are strongly associated with higher return risk contexts.
- Text-only modelling is useful but weaker than the fused structured + text model.

Table 2: Text-only NLP model performance (actual notebook outputs).

Metric	Value
Accuracy	0.3030
Precision (Return)	0.0194
Recall (Return)	0.7941
F1 (Return)	0.0380
ROC-AUC	0.6142
PR-AUC	0.0316

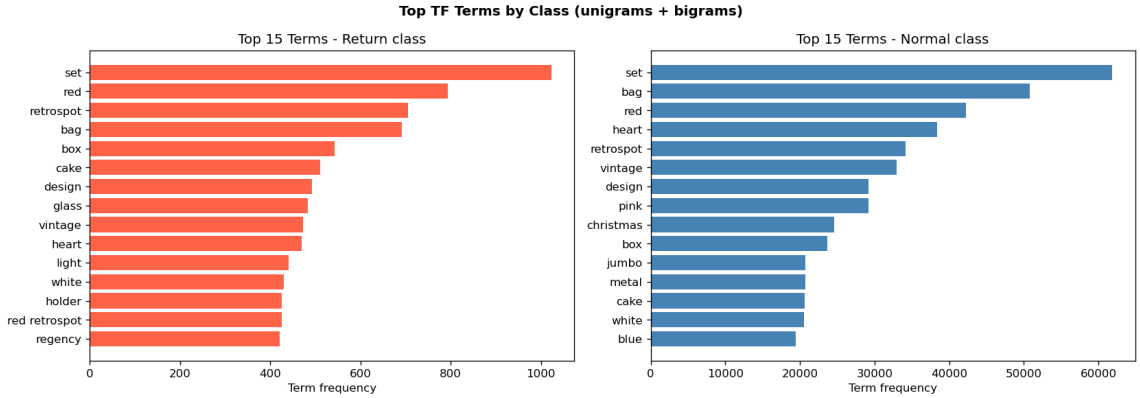

Figure 4: Top terms by return class versus normal class.

NLP output files include keyword coefficients and visualizations, such as `Tumushiime_nlp_keywords.csv`.

6 Deployment Architecture and Application Link

6.1 Architecture

The deployment design is self-contained and optimized for simple cloud hosting:

- **Application layer:** Streamlit app for single and batch predictions.
- **Inference engine:** Local in-app preprocessing and model scoring logic.
- **Model artefact:** Joblib bundle containing model, threshold, and metadata.
- **Deployment mode:** Streamlit Cloud hosts the interface and inference in one service.

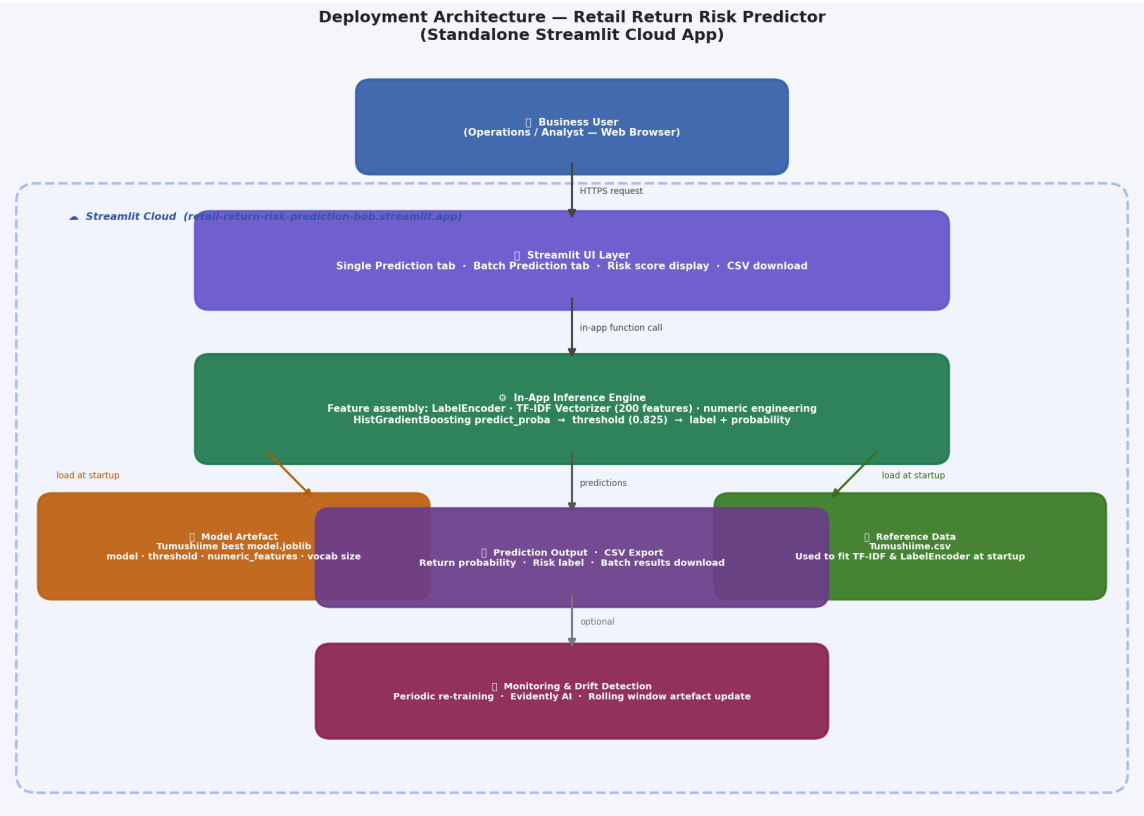

Figure 5: Deployment architecture for standalone Streamlit operational inference.

6.2 Running Application Link

The application link included in this report is:

- **Streamlit application link:** https://retail-return-risk-prediction-bob.streamlit.app/

Note: The Streamlit URL above is publicly accessible via Streamlit Cloud and runs end-to-end predictions directly in the app.

7 User Guidelines

7.1 How to use the web application

1. Open the Streamlit app URL: https://retail-return-risk-prediction-bob.streamlit.app/.
2. Check sidebar status and confirm standalone local inference is active.
3. Choose one workflow:
   - **Single Prediction tab:** Enter product description and transaction details, then click Predict.

- **Batch Prediction tab:** Upload CSV and run batch scoring.

4. Read outputs:
   - Return probability (continuous score)
   - Predicted class label (RETURN or Normal Sale)

7.2 Expected batch CSV columns

For best compatibility, include at least:

- Description
- UnitPrice
- Quantity
- Country

Optional fields accepted by defaults in UI:

- InvoiceMonth, InvoiceWeekday, InvoiceHour, CustomerID

7.3 Optional local run command (technical users)

- Clone/download the project files to a local machine with Python.
- Install dependencies: `pip install -r requirements.txt`
- Launch app: `streamlit run streamlit_app.py`

8 Ethical, Privacy, and Reliability Considerations

Responsible operationalization requires safeguards in three areas:

- **Ethics and fairness:** monitor subgroup error rates to reduce unjust impact.
- **Privacy and security:** apply data minimization, controlled access, and secure storage.
- **Reliability:** monitor drift and model quality over time, then retrain on schedule.

Predictions should support decisions, not replace policy and human oversight.

9 Conclusion

This project delivers a complete end-to-end retail return prediction solution: data preparation, feature engineering, model selection, NLP analysis, deployment scripts, and a working application interface. The final system is suitable for operational piloting and can be scaled with scheduled retraining, drift monitoring, and public hosting for wider access.

Submitted artefacts:

- Notebook: `Retail_Final2.ipynb`
- Clean data: `Tumushiime.csv`
- Model: `Tumushiime best model.joblib`
- Predictions: `Tumushiime predictions.csv`
- NLP outputs: keyword CSV and comparison figures
- Deployment script: `streamlit_app.py`
- Streamlit dependencies: `requirements.txt`